

Compiler-Emitted Nearest-Neighbor Systolic Arrays on FPGA

A Kung-style LU and one-sided Jacobi SVD case study, benchmarked against Vitis HLS

MS Thesis Proposal · Gary Pham · Drexel ECE

Advisor: Prof. Prawat Nagvajara · Draft prepared 2026-09-30 for winter term (Jan 27) start

1 Motivation

FPGA systolic arrays remain one of the most efficient substrates for dense linear algebra, but designers still hand-write RTL for each kernel or accept the area/latency overhead of high-level synthesis. Polyhedral-to-systolic compilers such as AutoSA [11] have shown that schedules can be generated automatically, yet the emitted arrays typically use long-range interconnect and non-streaming interfaces, which complicates system integration on modern AMD/Xilinx platforms where AXI4-Stream is the lingua franca [12, 13].

This thesis argues for the opposite constraint set: *strict nearest-neighbor connectivity and AXI4-Stream-compliant outputs by construction*. The claim is that this constraint pair, once baked into a shared PE template, is expressive enough to cover several important dense-linear-algebra kernels while yielding place-and-route results competitive with, or better than, Vitis HLS on the same C source.

2 Prior Work Positioning

FPGA implementations of SVD are dense in the literature. The Brent–Luk–Van Loan mesh [1] underlies most subsequent two-sided designs, including Hemkumar & Cavallaro’s complex-matrix CORDIC array [2], Ma et al.’s FPGA SVD processor [3], and Ahmedsaid & Amira’s improved systolic array [4]. One-sided (Hestenes-Jacobi) variants trade the two-sided rotation for better parallelism on rectangular matrices; representative work includes Wang & Zambreno’s floating-point Hestenes-Jacobi core [5], Karthikeyan’s one-bit-rotation CORDIC PEs [6], Kalaycıoğlu’s real-time two-sided design [7], the UCSB scalable engine with Maximum-Data-Sharing ordering [8], and a recent data-stream Jacobi architecture [9]. LU decomposition on systolic arrays traces back to Kung & Leiserson [10] and has similarly wide coverage.

The novelty of this thesis is **not** another SVD or LU accelerator. It is the compiler-plus-template flow, with LU and SVD serving as demonstrators driven from a shared PE template.

3 Contribution and Thesis Statement

A nearest-neighbor AXI4-Stream PE template, combined with a schedule-generation flow, can emit both a Kung-style LU decomposer and a one-sided Jacobi SVD engine from a single hardware template, matching or exceeding Vitis HLS on area $\times$ latency for both kernels on a Kintex-class target.

Concretely, the thesis will deliver:

1. A parameterized nearest-neighbor PE template with an AXI4-Stream boundary that respects UG1399 handshake semantics.
2. A schedule generator that maps LU and one-sided Jacobi SVD onto $N \times N$ meshes of that template.

3. Post-place-and-route benchmarks against Vitis HLS on identical C reference and against the closest prior FPGA-SVD numbers.
4. An open-source release of the flow, sufficient for reproducibility.

4 Repository Status (as of 2026-10-06)

Progress already in the working tree, ahead of the winter-term start:

- `kung_lu_decom/` — VHDL Kung-style LU on a 4×4 nearest-neighbor mesh with divide/multiply-add/register cells and a fixed-point divider. Simulation clean.
- `kung_lu_support/` — portable C harness that generates stimulus, extracts L/U from the hardware output stream, and verifies $A = L \cdot U$. Runs via `make run-sim`.
- `kung_svd/` — two architectural variants of one-sided Jacobi SVD selectable via a `USE_BLV` generic. The single-pair serializer (`svd_jacobi_top`) uses Gram reducer (`svd_gram.vhd`, with threshold-skip and auto-prescaled outputs), vectoring+rotate CORDIC (`svd_angle_cordic.vhd`), and inline Givens apply. The BLV grid (`svd_jacobi_blv`) runs $N/2$ parallel pair pipelines (`svd_pair_pipeline.vhd`) through a Brent–Luk–Van Loan round-robin schedule (auto-generated into `svd_blv_schedule_pkg.vhd` by `tools/gen_blv_schedule_pkg.py`). The AXI4-Stream wrapper (`svd_axi_stream.vhd`) remains unchanged; the `USE_BLV` generic chains through the hierarchy for easy A/B comparison.

Verified end-to-end in the open-source `nvc` VHDL simulator against NumPy golden references (`tools/sim_svd.sh -c {serial,blv} -n N -q`). Results below.

N	Serializer cycles	BLV cycles	Speedup	Verdict
4	1,159	604	$1.9\times$	both PASS
8	12,512	3,252	$3.85\times$	both PASS
16	137,545	17,764	$7.7\times$	FAIL (Q1.16 precision floor)

The $N=8$ result is $3.85\times$ of the theoretical $4\times$ ideal for $N/2$ parallel pair pipelines; $N=16$ is $7.7\times$ faster but shares the serializer’s Q1.16 rotation-noise floor. The remaining gap at $N=16$ is architectural but can be closed with a wider internal datapath; this is scoped as future work in the thesis.

Outstanding work before publishable results: (a) Vitis HLS baselines on the target board, (b) post-place-and-route numbers, and (c) the wider internal datapath to close $N=16$ at 2^{-10} tolerance.

5 Approach

5.1 PE template

The AXI4-Stream wrapper exposes `TVALID/TREADY/TLAST`, a small local register file, and a parameterized arithmetic core. LU uses a divide/subtract/multiply core (`Div_cell.vhd`, `MA_cell.vhd`, `M_cell.vhd`, `R_cell.vhd`). SVD uses a self-contained one-sided Jacobi pair pipeline (`svd_pair_pipeline.vhd`) composed of a 3-MAC Gram reducer, a two-stage vectoring+rotate CORDIC, and an inline Givens multiplier. The same pair pipeline is used by both the serializer (one instance) and the BLV grid ($N/2$ parallel instances).

5.2 SVD case study

Two architectural variants share the pair pipeline:

- **Serializer** (`svd_jacobi_top`): sequential row-cyclic schedule, one pair at a time. $O(N^3)$ cycles with SWEPS = N ; verified correct at $N \in \{4, 8\}$ against NumPy at 2^{-10} relative tolerance.
- **BLV grid** (`svd_jacobi_blv`): Brent–Luk–Van Loan round-robin schedule, $N/2$ disjoint column pairs rotated in parallel per step, $N-1$ steps per sweep. Verified correct at $N \in \{4, 8\}$ at 2^{-9} tolerance (matching published BLV convention [3, 2]); $3.85\times$ speedup at $N=8$ relative to the serializer, approaching the theoretical $4\times$ ideal.

Fixed-point precision is Q1.16 in 18-bit words throughout. The BLV schedule is auto-generated into `svd_blv_schedule_pkg.vhd` by `tools/gen_blv_schedule_pkg.py`, which also verifies disjointness and exhaustiveness.

5.3 Baseline and comparators

Comparator	Reference	Metric
Vitis HLS (same C, same board)	Internal base-line	LUT/FF/BRAM/DSP, $F_{\max}$, cycles
Ma et al. FPGA SVD processor	[3]	Reported cycles & area, normalized
UCSB scalable Jacobi engine	[8]	Throughput vs. bandwidth
DSB-Jacobi (data-stream)	[9]	BRAM footprint, streaming latency

6 Six-Month Schedule (Jan 27 – Jul 24, 2027)

Items marked “(done)” were completed ahead of the term in the pre-winter prototyping work.

Weeks	Milestone	Deliverable
0 (done)	Numerical golden reference, serializer, BLV grid, scale study at $N \in \{4, 8, 16\}$	<code>tools/sim_svd.sh</code> , <code>docs/scale_study.md</code> ; PASS at $N \in \{4, 8\}$ for both cores
1–3	Vivado + Vitis HLS baselines for LU and SVD on Kintex-7	Baseline table (LUT/FF/BRAM/DSP, $F_{\max}$, cycles)
4–7	Wider internal datapath to close $N=16$ at 2^{-10} tolerance; convergence verification on random 16×16 matrices	$N=16$ PASS; $\ A - U\Sigma V^T\ $ error plots
8–12	First post-P&R numbers on Kintex-7 for serializer + BLV at $N \in \{4, 8, 16\}$	Area/latency table vs. Vitis HLS and vs. Ma 2006
13–16	Extract shared PE/schedule template so LU and SVD emit from one flow	Compiler flow write-up
17–20	Full benchmark sweep; paper draft (target: FCCM/FPL 2027 or TRETTS)	Submitted paper draft; thesis chap- ters 1–4 in review
21–24	Thesis defense preparation; PhD application writing sample finalized	Defended thesis, tagged v1.0 open- source release

7 Risks and Mitigations

- **CORDIC accuracy at fixed-point.** Mitigation: 16-bit fraction starting point per Ma et al. [3]; widen only if reconstruction error exceeds 10^{-3} on 8×8 test matrices.
- **Convergence of the one-sided sweep.** Mitigation: the current design uses a fixed 8-sweep schedule; add off-diagonal-norm tracking to promote to an adaptive stop if convergence is uneven.
- **Scope creep into rectangular matrices or floating-point.** Mitigation: keep the thesis to square, fixed-point Q1.16; document Hestenes-Jacobi rectangular support as future work.
- **Board unavailability.** Mitigation: primary target Kintex-7 (already in the lab); simulation-only fallback path exists via the current xsim flow.
- **Vitis HLS producing surprisingly good baselines.** This is an acceptable outcome; the paper still stands if the compiler flow matches HLS with a fraction of the DSP count or with strictly nearest-neighbor placement, which HLS does not guarantee.

8 What I Am Asking For

1. Your read on whether this field — compiler-emitted nearest-neighbor systolic arrays with LU/SVD as demonstrators — is still workable as a thesis contribution given the density of prior FPGA-SVD work, or whether you would steer me toward an adjacent framing.
2. Confirmation of scope for a six-month MS thesis under your advisement.
3. Target-device sign-off (proposed: Kintex-7 already available; open to Zynq UltraScale+ if you prefer for the paper).
4. Agreement to co-author the resulting FCCM/FPL/TRETS submission.
5. A short kickoff meeting before winter term so weeks 1–3 land on the ground running.

References

References

- [1] R. P. Brent, F. T. Luk, and C. Van Loan, “Computation of the singular value decomposition using mesh-connected processors,” *Journal of VLSI and Computer Systems*, vol. 1, no. 3, pp. 242–270, 1985.
- [2] N. D. Hemkumar and J. R. Cavallaro, “A CORDIC processor array for the SVD of a complex matrix,” in *SVD and Signal Processing III*, 1994.
- [3] Y. Ma, M. Kaye, Q. Wang, et al., “An FPGA-based singular value decomposition processor,” in *Proc. IEEE CCECE*, 2006, pp. 1047–1050. doi:10.1109/CCECE.2006.277524.
- [4] A. Ahmedsaid, A. Amira, and A. Bouridane, “Improved SVD systolic array and implementation on FPGA,” in *Proc. IEEE FPT*, 2003, pp. 35–42. doi:10.1109/FPT.2003.1275730.
- [5] X. Wang and J. Zambreno, “An FPGA implementation of the Hestenes–Jacobi algorithm for singular value decomposition,” in *Proc. IEEE IPDPSW*, 2014, pp. 220–227. doi:10.1109/IPDPSW.2014.32.
- [6] C. Karthikeyan et al., “FPGA implementation of singular value decomposition,” *Int. Journal of Advanced Research in Computer Science*, 2013.
- [7] C. Kalaycıoğlu et al., “An FPGA implementation of singular value decomposition,” in *Proc. IEEE INISTA*, 2019. doi:10.1109/INISTA.2019.8778279.

- [8] W. Liu, D. Wang, and L. Peng, “A scalable FPGA engine for parallel acceleration of singular value decomposition,” in *Proc. IEEE ISQED*, 2020. [PDF](#).
- [9] Anon., “Design of a low-latency and parallelizable SVD dataflow architecture on FPGA,” arXiv:2511.12461, 2025. arxiv.org/abs/2511.12461.
- [10] H. T. Kung and C. E. Leiserson, “Systolic arrays (for VLSI),” in *Sparse Matrix Proc. 1978*, SIAM, 1979, pp. 256–282.
- [11] J. Wang, L. Guo, and J. Cong, “AutoSA: A polyhedral compiler for high-performance systolic arrays on FPGA,” in *Proc. ACM FPGA*, 2021, pp. 93–104. [doi:10.1145/3431920.3439292](https://doi.org/10.1145/3431920.3439292).
- [12] AMD Xilinx, “Vitis HLS User Guide (UG1399): AXI4-Stream Interfaces,” 2025. docs.amd.com/r/en-US/ug1399-vitis-hls.
- [13] Xilinx, “AXI Reference Guide (UG761),” 2017. [UG761 PDF](#).

Prepared by Gary Pham for review by Prof. Prawat Nagvajara. Repository: `/Users/garypham/Drexel/FPGA`. This document is a working proposal; scope and schedule will be revised after our kickoff meeting.